

UD5 - Desarrollo de servicios web

Alumno: Francisco Pérez Ruiz **Asignatura:** Desarrollo Web en Entorno Servidor (DWES)

Introducción

En esta unidad damos un salto importante: pasamos de "hacer cosas con PHP" a empezar a entender **cómo se comunica realmente un navegador con un servidor**. Para eso aparecen el protocolo HTTP, las cabeceras, el JSON, las sesiones, las cookies y, ya al final, un sistema de autenticación de usuarios con SQLite y un CRUD completo en arquitectura MVC.

¿Qué es HTTP?

El **Protocolo de Transferencia de Hipertexto (HTTP)** es la base de la World Wide Web. Su función es permitir el intercambio de información entre un cliente (por ejemplo, el navegador) y un servidor.

Cuando entro a una página web pasa lo siguiente:

1. El cliente envía una **solicitud** al servidor pidiendo un recurso.
2. El servidor procesa la petición y devuelve una **respuesta** con un código de estado (**200 OK**, **404 Not Found**, **500...**) y el contenido pedido.

HTTP es un protocolo "sin estado": cada petición es independiente, el servidor no recuerda nada de las anteriores. Para sortear esa limitación existen las **sesiones** y las **cookies**, que se ven en esta unidad.

Una solicitud HTTP contiene, a grandes rasgos:

1. La versión del protocolo.
2. La URL del recurso.
3. El método HTTP (**GET**, **POST**, **PUT**, **DELETE...**).
4. Las cabeceras (**headers**).
5. Un cuerpo opcional con datos.

P1 - Subida de archivos

Primer ejercicio: un formulario HTML que permite subir un archivo, y un script PHP que lo recibe, lo guarda en la carpeta `uploads/` y comprueba el tamaño máximo permitido. Si el archivo pesa más de 2MB lo rechaza.

Formulario `subir.html`:

`subir.html` · líneas 1-15

```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4     <meta charset="UTF-8">
5     <title>Subir Archivo</title>
6 </head>
7 <body>
8     <h1>Subir un archivo</h1>
9     <form action="subir.php" method="POST" enctype="multipart/form-data">
10         <label for="archivo">Selecciona un archivo para subir:</label>
11         <input type="file" name="archivo" id="archivo" required><br><br>
12         <input type="submit" value="Subir Archivo">
13     </form>
14 </body>
15 </html>
```

Procesador `subir.php` (validación y carpeta):

subir.php · líneas 1-25

```
1 <?php
2 // Verificar si el formulario ha sido enviado
3 if ($_SERVER['REQUEST_METHOD'] == 'POST') {
4     // Comprobar si el archivo ha sido cargado sin errores
5     if (isset($_FILES['archivo']) && $_FILES['archivo']['error'] == 0) {
6
7         // Información del archivo
8         $nombre_archivo = $_FILES['archivo']['name'];
9         $tipo_archivo = $_FILES['archivo']['type'];
10        $tmp_archivo = $_FILES['archivo']['tmp_name'];
11        $tamaño_archivo = $_FILES['archivo']['size'];
12
13        // Definir el directorio de destino
14        $directorio_destino = 'uploads/'; // Asegúrate de crear esta carpeta y darle permisos de escritura
15
16        // Verificar si el directorio existe, si no, crear el directorio
17        if (!is_dir($directorio_destino)) {
18            mkdir($directorio_destino, 0777, true); // 0777 otorga permisos de lectura, escritura y ejecución a todos
19            echo "Carpeta de destino creada.<br>";
20        }
21
22        // Comprobar el tamaño del archivo (por ejemplo, limitar a 2MB)
23        //probar un cambio a tamaño muy pequeño para ver el mensaje de error
24        // if ($tamaño_archivo <= 2 ) {
25        if ($tamaño_archivo <= 2 * 1024 * 1024) {
```

Procesador **subir.php** (mover archivo y comprobar tamaño):

subir.php · líneas 27-41

```
27         if (move_uploaded_file($tmp_archivo, $directorio_destino . $nombre_archivo)) {
28             echo "El archivo '$nombre_archivo' ha sido subido exitosamente.";
29         } else {
30             echo "Error al mover el archivo al directorio de destino.";
31         }
32     } else {
33         echo "El archivo es demasiado grande. El tamaño máximo permitido es 2MB.";
34     }
35 } else {
36     echo "No se ha subido ningún archivo o ha ocurrido un error en la carga.";
37 }
38 } else {
39     echo "Por favor, use el formulario para subir un archivo.";
40 }
41 ?>
```

Formulario en el navegador:

Subir un archivo

Selecciona un archivo para subir: Ningún archivo seleccionado

Al subir un fichero se crea automáticamente la carpeta `uploads/` si no existe, y se valida que no supere los 2MB.

P2 - Cabeceras HTTP

Las **cabeceras** (`headers`) son los metadatos que viajan junto con la petición o la respuesta HTTP. Sirven para indicar tipo de contenido, política de caché, redirecciones, errores, etc.

Ejemplo 1 - Cabecera JSON

`header('Content-Type: application/json')` indica al navegador que la respuesta es JSON. Después convierto un array PHP a JSON con `json_encode()`.

Ejemplo1.php · líneas 1-12

```
1 <?php
2     header('Content-Type: application/json');
3
4     $data = [
5         "nombre" => "Francisco",
6         "edad" => 25,
7         "ciudad" => "Madrid"
8     ];
9
10
11     echo json_encode($data);
12 ?>
```

Dar formato al texto ☐

```
{"nombre": "Francisco", "edad": 25, "ciudad": "Madrid"}
```

Ejemplo 2 - Devolver un error 404

Sirve para devolver explícitamente una respuesta **404 No encontrado** desde PHP, sin que la página realmente exista.

Ejemplo2.php · líneas 1-7

```
1 <?php
2 header("HTTP/1.1 404 No encontrado");
3 echo "Página no encontrada";
4
5
6 ?>
7
```

Ejemplo 3 - Cabeceras combinadas y `headers_list()`

Aquí mando varias cabeceras (`Content-Type`, `Cache-Control`, `Pragma`) y después imprimo todas las cabeceras que se van a enviar con `headers_list()`.

Ejemplo3.php · líneas 1-19

```
1 <?php
2
3
4 //header('Location: ejemplo1.php', true, 302);
5 // o permanente:
6 // header('Location: https://ejemplo.com', true, 301);
7 header('Content-Type: application/json; charset=utf-8');
8 header('Cache-Control: no-store, no-cache, must-revalidate, max-age=0');
9 header('Pragma: no-cache');
10
11
12 echo print_r(headers_list());
13 exit; // importante
14
15
16
17
18 ?>
19
```

```
Array
(
    [0] => X-Powered-By: PHP/8.2.12
    [1] => Content-Type: application/json; charset=utf-8
    [2] => Cache-Control: no-store, no-cache, must-revalidate, max-age=0
    [3] => Pragma: no-cache
)
```

Ejemplo 4 - Caducidad con Expires

Práctica con la cabecera **Expires**, que indica cuándo caduca un recurso. Aquí pruebo con expiraciones distintas (3 horas, 1 año).

Ejemplo4.php · líneas 1-22

```
1 <?php
2
3
4 header("Content-Type: application/json; charset=utf-8 ");
5 header("Expires: Thu, 01 Dec 1994 16:00:00 GMT");
6
7 $now = time();
8 $horas3 = gmdate("D, d M Y H:i:s", $now + 60 * 60 * 3) . " GMT";
9 header("Expires: {$horas3}");
10
11
12 $anyo1 = gmdate("D, d M Y H:i:s", $now + 365 * 86400) . " GMT";
13 // Nota: 86400 segundos = 1 día
14 header("Expires: {$anyo1}");
15
16
17
18 echo print_r(headers_list());
19
20
21 ?>
22
```

```
Array
(  
  [0] => X-Powered-By: PHP/8.2.12  
  [1] => Content-Type: application/json; charset=utf-8  
  [2] => Expires: Wed, 09 Jun 2027 15:39:40 GMT  
)  
1
```

P3 - Trabajar con JSON

JSON es el formato estándar para intercambiar datos entre cliente y servidor. Aquí lo veo en su forma más sencilla: un formulario HTML que envía datos por POST, y un PHP que los recibe, valida y devuelve la respuesta en JSON.

Formulario [index.html](#):


```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4     <meta charset="UTF-8">
5     <title>Enviar Datos en JSON sin JavaScript</title>
6 </head>
7 <body>
8     <h1>Formulario para Enviar Datos en JSON</h1>
9     <form action="procesar.php" method="POST">
10         <label for="nombre">Nombre:</label>
11         <input type="text" id="nombre" name="nombre" required><br><br>
12
13
14         <label for="email">Email:</label>
15         <input type="email" id="email" name="email" required><br><br>
16
17
18         <input type="submit" value="Enviar">
19     </form>
20 </body>
21 </html>
```

Procesador `procesar.php`:

```
1 <?php
2 // Verificar si se han enviado datos a través de POST
3 if ($_SERVER['REQUEST_METHOD'] == 'POST') {
4     // Obtener los datos enviados a través del formulario
5     $nombre = isset($_POST['nombre']) ? $_POST['nombre'] : null;
6     $email = isset($_POST['email']) ? $_POST['email'] : null;
7
8
9     // Comprobar si ambos campos han sido enviados
10    if ($nombre && $email) {
11        // Crear un array con los datos para la respuesta
12        $respuesta = [
13            'status' => 'success',
14            'mensaje' => 'Datos recibidos correctamente.',
15            'datos' => [
16                'nombre' => $nombre,
17                'email' => $email
18            ]
19        ];
20    } else {
21        // Responder con un error si los datos no están completos
22        $respuesta = [
23            'status' => 'error',
24            'mensaje' => 'Faltan datos en la solicitud.'
25        ];
26    }
27
28
29    // Establecer la cabecera Content-Type a application/json
30    header('Content-Type: application/json');
31
32
33    // Enviar la respuesta en formato JSON
34    echo json_encode($respuesta);
35 }
36 ?>
37
```

Formulario en el navegador:

Formulario para Enviar Datos en JSON

Nombre:

Email:

Cuando el formulario se envía, el servidor responde con un JSON tipo:

```
{ "status": "success", "mensaje": "Datos recibidos correctamente.", "datos": { ...  
} }
```

P4 - Redirecciones HTTP

Las redirecciones se hacen mandando una cabecera **Location** junto con un código de estado. Cada código tiene un significado:

Código	Significado
301	Redirección permanente
302	Redirección temporal
303	Redirección después de un POST
307	Redirección temporal (HTTP/1.1)
308	Redirección permanente (HTTP/1.1)

Formulario para elegir el tipo de redirección:

index.html · líneas 1-21

```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4     <meta charset="UTF-8">
5     <title>Seleccionar Cabecera de Redirección</title>
6 </head>
7 <body>
8     <h1>Elija el tipo de cabecera de redirección</h1>
9     <form action="redireccion.php" method="POST">
10         <label for="cabecera">Selecciona el tipo de cabecera de redirección:</label><br><br>
11         <select name="cabecera" id="cabecera">
12             <option value="301">301 - Redirección permanente</option>
13             <option value="302">302 - Redirección temporal</option>
14             <option value="303">303 - Redirección después de una operación POST</option>
15             <option value="307">307 - Redirección temporal (HTTP/1.1)</option>
16             <option value="308">308 - Redirección permanente (HTTP/1.1)</option>
17         </select><br><br>
18         <input type="submit" value="Redirigir">
19     </form>
20 </body>
21 </html>
```

Procesador que lanza la redirección:

redireccion.php · líneas 1-22

```
1 <?php
2 // Verificar si el formulario ha sido enviado
3 if ($_SERVER['REQUEST_METHOD'] == 'POST') {
4     // Obtener la cabecera seleccionada por el usuario
5     $cabecera = isset($_POST['cabecera']) ? $_POST['cabecera'] : null;
6
7
8     // Definir la URL de redirección
9     $url_redireccion = 'https://www.diariocordoba.com'; // URL de destino para la redirección
10
11
12     // Validar que se ha seleccionado una cabecera válida
13     if ($cabecera && in_array($cabecera, ['301', '302', '303', '307', '308'])) {
14         // Enviar la cabecera de redirección correspondiente
15         header("Location: $url_redireccion", true, $cabecera);
16         exit(); // Terminar la ejecución del script después de la redirección
17     } else {
18         echo "Selecciona un tipo de cabecera válido.";
19     }
20 }
21 ?>
22
```

Formulario en el navegador:

Elija el tipo de cabecera de redirección

Selecciona el tipo de cabecera de redirección:

301 - Redirección permanente ▼

Redirigir

P5 - Sesiones

Como HTTP es sin estado, para "recordar" cosas entre peticiones se usan las **sesiones**. Una sesión es un array asociativo (`$_SESSION`) que se guarda en el servidor y se identifica con una cookie que envía el navegador.

Funciones que se usan:

- `session_start()` para iniciar o recuperar la sesión.
- `$_SESSION['clave'] = 'valor'` para guardar datos.
- `unset($_SESSION['clave'])` para borrar una variable concreta.
- `session_unset()` borra todas las variables.
- `session_destroy()` destruye la sesión entera.

Ejemplo 1 - Iniciar sesión y guardar un usuario

ejemplo1.php · líneas 1-8

```
1
2 <?php
3 session_start();
4
5 $_SESSION['usuario'] = 'FranciscoPerez';
6
7 echo "Sesión iniciada. Usuario: " . $_SESSION['usuario'];
8 ?>
```

Sesión iniciada. Usuario: FranciscoPerez

Ejemplo completo - Contador de visitas y fechas

Este programa mezcla varios casos: crear variables de sesión, eliminarlas, llevar un contador de visitas y guardar la fecha de cada visita.

Primera parte (crear, mostrar y eliminar variables):

```
14  <?php
15  //DESACTivo Los warnings para que no molesten en la práctica
16  error_reporting(E_ALL & ~E_NOTICE & ~E_WARNING);
17  // Inicia una nueva sesión o recupera la sesión existente
18  session_start();
19
20  // -----
21  // Ejemplo 1: Crear y mostrar una variable de sesión
22  // -----
23
24  // Crear una variable de sesión llamada 'newsession' y asignarle un valor
25  $_SESSION["newsession"] = "Mi primera sesión";
26
27  // Mostrar el valor de la variable de sesión 'newsession'
28  echo "<p><strong>Valor de la sesión 'newsession':</strong> " . $_SESSION["newsession"] . "</p>";
29
30  // -----
31  // Ejemplo 2: Crear, mostrar y eliminar una variable de sesión
32  // -----
33
34  // Crear otra variable de sesión llamada 'sessiontwo'
35  $_SESSION["sessiontwo"] = "Hello, I am Rafa";
36
37  // Mostrar el valor de 'sessiontwo' antes de eliminarla
38  echo "<p><strong>Antes de borrar 'sessiontwo':</strong> " . $_SESSION["sessiontwo"] . "</p>";
39
40  // Eliminar la variable de sesión 'sessiontwo' utilizando unset()
41  unset($_SESSION["sessiontwo"]);
42
43  // Intentar mostrar la variable de sesión después de eliminarla (generará un aviso)
44  echo "<p><strong>Después de borrar 'sessiontwo':</strong> " . ($_SESSION["sessiontwo"] ?? "La sesión ha sido eliminada") . "</p>";
45
46  // -----
47  // Ejemplo 3: Contador de visitas utilizando sesiones
48  // -----
49
50  // Verificar si la variable de sesión 'visitas' ya existe
```

Segunda parte (contador de visitas, fechas, botón para destruir):

```

60     echo "<p><strong>Número de visitas:</strong> " . $_SESSION['visitas'] . "</p>";
61
62     // -----
63     // Ejemplo 4: Almacenar la fecha y hora de cada visita
64     // -----
65
66     // Añadir la fecha y hora actuales al array de visitas
67     $_SESSION['visitasFecha'][] = date('d-m-Y H:i:s');
68
69     // Mostrar todas las fechas de visitas almacenadas
70     echo "<p><strong>Fechas de visitas:</strong></p>";
71     echo "<ul>";
72     foreach ($_SESSION['visitasFecha'] as $fecha) {
73         echo "<li>" . $fecha . "</li>";
74     }
75     echo "</ul>";
76
77     // -----
78     // Ejemplo 5: MOSTRANDO LAS VARIABLES DE SESIÓN CREADAS
79     // -----
80
81     if (!empty($_SESSION)) {
82         echo "<h2>Variables de sesión creadas:</h2>";
83         echo "<ul>";
84         foreach ($_SESSION as $clave => $valor) {
85             echo "<li><strong>$clave:</strong> $valor</li>";
86         }
87         echo "</ul>";
88     } else {
89         echo "<h2>No hay variables de sesión creadas.</h2>";
90     }
91
92     //eliminar la sesión completamente (descomentar la línea siguiente si se desea)
93     // session_destroy();
94
95     // -----
96     // Ejemplo 5: MOSTRANDO LAS VARIABLES DE SESIÓN TRAS DESTRUIRLA
97     // -----
98
99     //Crea un botón para recargar la página
100    echo '<form method="post"><button type="submit" name="reload">Recargar página</button></form>';
101
102    //Crea un botón para destruir la sesión
103    echo '<form method="post"><button type="submit" name="destroy">Destruir sesión</button></form>';
104    if (isset($_POST['destroy'])) {
105        session_destroy();
106        // Recargar la página para reflejar los cambios
107        header("Location: " . $_SERVER['PHP_SELF']);
108        exit();
109    }
110

```

Salida en el navegador:

Usando Sesiones

Valor de la sesión 'newsession': Mi primera sesión

Antes de borrar 'sessiontwo': Hello, I am Rafa

Después de borrar 'sessiontwo': La sesión ha sido eliminada

Número de visitas: 0

Fechas de visitas:

- 09-06-2026 17:39:44

Variables de sesión creadas:

- **usuario:** FranciscoPerez
- **newsession:** Mi primera sesión
- **visitas:** 0
- **visitasFecha:** Array

Recargar página

Destruir sesión

P6 - Cookies

Las **cookies** son pequeños trozos de información que el servidor guarda en el navegador y que se envían en cada petición. A diferencia de las sesiones, viven en el cliente.

Para crear o modificar una cookie se usa `setcookie(nombre, valor, expiración, ruta)`. Para borrarla se pone una fecha de expiración en el pasado.

Aquí pruebo varios casos en un mismo archivo: guardar un puntaje, modificar un nivel, eliminar cookies, guardar volumen, idioma, y un foreach que borra todas las cookies almacenadas.

Primera parte (puntaje, nivel, eliminar):

```
1 <?php
2 // Guardar el puntaje del jugador en una cookie válida por 1 día
3 setcookie("puntaje_jugador", "1500", time() + 86400, "/");
4
5 // Acceder al puntaje
6 if (isset($_COOKIE['puntaje_jugador'])) {
7     echo "Tu puntaje actual es: " . $_COOKIE['puntaje_jugador'] ;
8 }
9
10 echo "<br>";
11
12 // Modificar el nivel alcanzado por el jugador
13 setcookie("nivel_jugador", "2", time() + 86400, "/");
14
15 // Cambiar el nivel a 3
16 setcookie("nivel_jugador", "3", time() + 86400, "/");
17
18 echo "Has avanzado al nivel: " . $_COOKIE['nivel_jugador'];
19
20 echo "<br>";
21 // Para eliminar la cookie, se establece una fecha de expiración en el pasado
22 setcookie("puntaje_jugador", "", time() - 3600, "/");
23
24 echo "Puntaje reiniciado.";
25 // Guardar configuración de volumen del juego (solo válida durante la sesión)
```

Segunda parte (volumen, idioma, borrar todas y tabla de cookies):

```

26 setcookie("volumen_audio", "80", 0, "/");
27
28 if (isset($_COOKIE['volumen_audio'])) {
29     echo "Volumen actual: " . $_COOKIE['volumen_audio'] . "%";
30 }
31
32 // Guardar preferencia de idioma en una cookie válida por 30 días
33 setcookie("idioma", "es", time() + (30 * 24 * 60 * 60), "/");
34
35 echo "El idioma seleccionado es: " . $_COOKIE['idioma'];
36
37
38 foreach ($_COOKIE as $nombre => $valor) {
39     setcookie($nombre, "", time() - 3600, "/");
40 }
41
42 echo "Todas las cookies han sido eliminadas. ¡Juego reiniciado!";
43
44
45 ?>
46
47 <!-- Tabla que muestra todas las cookies almacenadas -->
48 <table border="1" cellpadding="5" cellspacing="0">
49     <thead>
50         <tr>
51             <th>Variable</th>
52             <th>Valor</th>
53         </tr>
54     </thead>
55     <tbody>
56         <?php
57             // Itera a través de todas las cookies almacenadas y las muestra en la tabla
58             foreach ($_COOKIE as $variable => $valor) {
59                 echo "<tr>";
60                 echo "<td>" . htmlspecialchars($variable) . "</td>"; // Muestra el nombre de la cookie
61                 echo "<td>" . htmlspecialchars($valor) . "</td>"; // Muestra el valor de la cookie
62                 echo "</tr>";
63             }
64         ?>
65     </tbody>
66 </table>

```

Salida en el navegador:

Warning: Undefined array key "nivel_jugador" in C:\xampp\htdocs\DWES_FranciscoPerez\UD5_HerramientasWeb\P6_Cookie\Ejemplo1.php on line 18

Has avanzado al nivel:

Puntaje reiniciado.

Warning: Undefined array key "idioma" in C:\xampp\htdocs\DWES_FranciscoPerez\UD5_HerramientasWeb\P6_Cookie\Ejemplo1.php on line 35

El idioma seleccionado es: Todas las cookies han sido eliminadas. ¡Juego reiniciado!

Variable	Valor
PHPSESSID	n8m077sva4011ulajvf2s57eld

P7 - Autenticación

Este apartado son **cinco mini-proyectos** que van subiendo de complejidad: desde un login pelado contra una base de datos SQLite hasta un CRUD completo con sesiones, autenticación y arquitectura MVC.

1. Login básico (texto plano)

Primera versión muy básica. Crea una tabla en SQLite con (**user**, **pass**), inserta un usuario **admin/admin**, y un script **comprueba_login.php** que compara directamente lo que envía el usuario contra la base de datos.

Crear la tabla y meter el usuario inicial:

```
1 <?php
2 // Ruta donde se guardará la base de datos SQLite
3 $db_path = 'database.sqlite';
4
5 try {
6     // Conexión a la base de datos SQLite (se crea automáticamente si no existe)
7     $pdo = new PDO('sqlite:' . $db_path);
8
9     // Configuración para lanzar excepciones en caso de errores
10    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
11
12    // SQL para crear la tabla ud5__usuarios_pass
13    $createTableSQL = "
14    CREATE TABLE IF NOT EXISTS ud5__usuarios_pass (
15        ID INTEGER PRIMARY KEY AUTOINCREMENT,
16        user VARCHAR(20) NOT NULL,
17        pass TEXT NOT NULL
18    )";
19
20    // Ejecutar la creación de la tabla
21    $pdo->exec($createTableSQL);
22
23    echo "Tabla 'ud5__usuarios_pass' creada con éxito.<br>";
24
25    // Insertar un usuario en la tabla
26    $insertSQL = "
27    INSERT INTO ud5__usuarios_pass (user, pass)
28    VALUES (:user, :pass)";
29
30    // Preparar y ejecutar la inserción de datos
31    $stmt = $pdo->prepare($insertSQL);
32    $stmt->execute([
33        ':user' => 'admin',
34        ':pass' => 'admin'
35    ]);
36
37    echo "Usuario 'admin' insertado con éxito.";
38 } catch (PDOException $e) {
39     // Mostrar el error si ocurre
40     echo "Error en la creación de la tabla o inserción de datos: " . $e->getMessage();
41 }
42 ?>
```

Formulario **login.php**:

```
1 <head>
2 <meta charset="utf-8">
3 <title>Login</title>
4 </head>
5
6 <body>
7
8 <h2>Sistema Login 1</h2>
9 <p>Esta práctica es un sistema de login </p>
10
11 <form action="comprueba_login.php" method="post">
12
13 <table>
14     <tr>
15         <td class="izq"> Login </td>
16         <td class="der"><input type="text" name="login"> </td>
17     </tr>
18     <tr>
19         <td class="izq"> Contraseña </td>
20         <td class="der"><input type="password" name="password"> </td>
21     </tr>
22     <tr>
23         <td colspan="2"><input type="submit" name="enviar" value="Login"> </td>
24     </tr>
25 </table>
26
27 </form>
28
29 </body>
30 </html>
```

Validador `comprueba_login.php`:

```
8 <?php
9 // Ruta a la base de datos SQLite
10 $db_path = 'database.sqlite';
11
12 try {
13     // Establecemos conexión con SQLite usando PDO
14     $base = new PDO('sqlite:' . $db_path);
15     $base->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
16
17     // Validamos las entradas
18     if (!isset($_POST['login']) || !isset($_POST['password'])) {
19         die("Error: Debes proporcionar usuario y contraseña.");
20     }
21
22     // Recogemos y preparamos los datos
23     $login = $_POST["login"];
24     $password = $_POST["password"];
25
26     // Definimos la consulta SQL con marcadores
27     $sql = "SELECT * FROM ud5__usuarios_pass WHERE user = :login AND pass = :password";
28     $resultado = $base->prepare($sql);
29
30     // Asociamos los valores
31     $resultado->bindValue(":login", $login);
32     $resultado->bindValue(":password", $password);
33
34     // Ejecutamos la consulta
35     $resultado->execute();
36
37     // Obtenemos registro
38     $registro = $resultado->fetch(PDO::FETCH_ASSOC);
39
40     if ($registro) {
41         // Usuario existe
42         echo "<h2>¡Adelante!</h2>";
43         session_start();
44         $_SESSION["usuario"] = $login;
45         header("Location: usuarios_registrados.php");
46         exit();
47     } else {
48         // Usuario NO existe
49         echo "Usuario o contraseña incorrectos. Redirigiendo...";
50         header("refresh:2; url=login.php");
51         exit();
52     }
53
54 } catch (Exception $e) {
55     die("Error: " . $e->getMessage());
56 }
57 ?>
```

Login en el navegador:

Sistema Login 1

Esta práctica es un sistema de login

Login	
Contraseña	
Login	

Nota: este ejemplo es solo educativo. **Nunca** se debe guardar la contraseña en claro en la base de datos. Esto se arregla en el siguiente apartado.

2. Login con hash

Aquí la contraseña ya no se guarda en texto plano. Al crear el usuario se usa `password_hash()`, y al hacer login se comprueba con `password_verify()`. Esa es la forma correcta de manejar contraseñas en PHP.

Crear la BD y el usuario con contraseña hasheada:


```
1 <?php
2 // Ruta y nombre de la base de datos SQLite
3 $db_file = 'database.sqlite';
4
5 try {
6     // Conexión a la base de datos SQLite
7     $db = new PDO("sqlite:" . $db_file);
8     $db->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
9
10    // Crear la tabla usuarios si no existe
11    $sql = "CREATE TABLE IF NOT EXISTS usuarios (
12            id INTEGER PRIMARY KEY AUTOINCREMENT,
13            usuario TEXT NOT NULL UNIQUE,
14            password TEXT NOT NULL
15        )";
16    $db->exec($sql);
17
18    // Insertar un usuario inicial (si no existe)
19    $usuario = 'admin';
20    $password = password_hash('admin', PASSWORD_DEFAULT); // Encriptar la contraseña
21    $stmt = $db->prepare("INSERT INTO usuarios (usuario, password) VALUES (:usuario, :password)");
22    $stmt->bindValue(':usuario', $usuario);
23    $stmt->bindValue(':password', $password);
24    $stmt->execute();
25
26    echo "Base de datos creada y usuario inicial insertado.";
27
28 } catch (PDOException $e) {
29     die("Error al crear la base de datos: " . $e->getMessage());
30 }
31 ?>
```

Procesador de login con `password_verify`:

```
1 <?php
2 session_start(); // Iniciar sesión
3 $db_file = 'database.sqlite';
4
5 try {
6     // Conectar a la base de datos SQLite
7     $db = new PDO("sqlite:" . $db_file);
8     $db->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
9
10    // Obtener los datos del formulario
11    $usuario = htmlentities($_POST['usuario']);
12    $password = htmlentities($_POST['password']);
13
14    // Buscar el usuario en la base de datos
15    $stmt = $db->prepare("SELECT * FROM usuarios WHERE usuario = :usuario");
16    $stmt->bindValue(':usuario', $usuario);
17    $stmt->execute();
18
19    // Comprobar si el usuario existe
20    if ($fila = $stmt->fetch(PDO::FETCH_ASSOC)) {
21        // Verificar la contraseña
22        if (password_verify($password, $fila['password'])) {
23            echo "<h2>¡Bienvenido, $usuario!</h2>";
24            $_SESSION['usuario'] = $usuario;
25            header("Location: usuarios_registrados.php");
26        } else {
27            echo "<h3>Contraseña incorrecta. <a href='index.php'>Volver a intentar</a></h3>";
28        }
29    } else {
30        echo "<h3>Usuario no encontrado. <a href='index.php'>Volver a intentar</a></h3>";
31    }
32 } catch (PDOException $e) {
33     die("Error en la conexión: " . $e->getMessage());
34 }
35 ?>
```

Página de login:

2. Login Hash. Iniciar Sesión

Usuario:

Contraseña:

3. Login + Productos

Tercer paso: una vez logueado, el usuario tiene acceso a una zona protegida. Aparece `funciones.php` con funciones de utilidad como `comprobarSesion()` para que las páginas protegidas redirijan al login si no hay sesión activa.

`funciones.php`:

`funciones.php` · líneas 1-15

```
1 <?php
2 function conectar_bd() {
3     $db_file = 'database.sqlite';
4     return new PDO("sqlite:" . $db_file);
5 }
6
7 function verificar_usuario($usuario, $password) {
8     $db = conectar_bd();
9     $stmt = $db->prepare("SELECT password FROM usuarios WHERE usuario = ?");
10    $stmt->execute([$usuario]);
11    $fila = $stmt->fetch(PDO::FETCH_ASSOC);
12
13    return $fila && password_verify($password, $fila['password']);
14 }
15 ?>
```

Página inicial del proyecto:

Bienvenido al Sistema Web

[Iniciar sesión](#)

[Productos](#)

[Contacto](#)

4. CRUD paginado

Aquí ya hay un CRUD completo de productos: listar, añadir, editar y eliminar, con paginación. Todo va contra SQLite y solo accesible si hay sesión activa.

config.php (conexión central a la BD):

```
1 <?php
2 // Nombre del fichero SQLite
3 define("DB_NAME", "database.sqlite");
4
5 try {
6     // Conexión con SQLite3 (estilo orientado a objetos)
7     $db = new SQLite3(DB_NAME);
8
9     // Crear tabla de usuarios si no existe
10    $db->exec("CREATE TABLE IF NOT EXISTS usuarios (
11        id INTEGER PRIMARY KEY AUTOINCREMENT,
12        usuario TEXT NOT NULL UNIQUE,
13        password TEXT NOT NULL
14    )");
15
16    // Crear tabla de productos si no existe
17    $db->exec("CREATE TABLE IF NOT EXISTS productos (
18        id INTEGER PRIMARY KEY AUTOINCREMENT,
19        nombre TEXT NOT NULL,
20        descripcion TEXT,
21        precio REAL NOT NULL,
22        stock INTEGER NOT NULL
23    )");
24
25    // Comprobar si existe usuario admin
26    $resultado = $db->query("SELECT COUNT(*) AS total FROM usuarios WHERE usuario = 'admin'");
27    $fila = $resultado->fetchArray(SQLITE3_ASSOC);
28
29    if ((int)$fila['total'] === 0) {
30        // Crear usuario admin con hash
```

Página inicial:

Fatal error: Uncaught Error: Class "SQLite3" not found in C:\xampp\htdocs\DWES_FranciscoPerez\UD5_HerramientasWeb\P7_Autenticacion\4_CrudPaginado\config.php:7 Stack trace: #0 C:\xampp\htdocs\DWES_FranciscoPerez\UD5_HerramientasWeb\P7_Autenticacion\4_CrudPaginado\index.php(3): require_once() #1 {main} thrown in C:\xampp\htdocs\DWES_FranciscoPerez\UD5_HerramientasWeb\P7_Autenticacion\4_CrudPaginado\config.php on line 7

5. CRUD paginado con arquitectura MVC

La versión final: el mismo CRUD pero reorganizado en **Modelo-Vista-Controlador**. Hay un `index.php` que actúa como front controller, controladores (`AuthController`, `ProductController`), modelos (`User`, `Product`) y vistas separadas (`auth`, `layout`, `product`).

Front controller `index.php`:

index.php · líneas 1-40

```
1 <?php
2 declare(strict_types=1);
3
4 require_once __DIR__ . '/config.php';
5 require_once __DIR__ . '/models/User.php';
6 require_once __DIR__ . '/models/Product.php';
7 require_once __DIR__ . '/controllers/AuthController.php';
8 require_once __DIR__ . '/controllers/ProductController.php';
9
10
11 if (!file_exists(__DIR__ . '/data/app.sqlite')) {
12     require_once __DIR__ . '/create_db.php';
13 }
14 ensureSession();
15
16 $pdo = getPdo();
17
18 $controllerName = $_GET['c'] ?? null;
19 $action = $_GET['a'] ?? null;
20
21 // Si no hay usuario autenticado, forzamos ir al login
22 if (!isset($_SESSION['user']) && !($controllerName === 'auth' && in_array($action, ['login', 'doLogin'], true))) {
23     $controllerName = 'auth';
24     $action = 'login';
25 }
26
27 // Valores por defecto
28 if ($controllerName === null) {
29     $controllerName = isset($_SESSION['user']) ? 'product' : 'auth';
30 }
31 if ($action === null) {
32     $action = ($controllerName === 'auth') ? 'login' : 'index';
33 }
34
35 // Crear instancias de controladores
36 $authController = new AuthController($pdo);
37 $productController = new ProductController($pdo);
38
39 switch ($controllerName) {
40     case 'auth':
```

Modelo `User`:

```
1 <?php
2 declare(strict_types=1);
3
4 class User
5 {
6     public int $id;
7     public string $username;
8     public string $password_hash;
9
10    public static function findByUsername(PDO $pdo, string $username): ?User
11    {
12        $stmt = $pdo->prepare('SELECT * FROM users WHERE username = :u LIMIT 1');
13        $stmt->execute([':u' => $username]);
14        $row = $stmt->fetch(PDO::FETCH_ASSOC);
15
16        if (!$row) {
17            return null;
18        }
19
20        $user = new User();
21        $user->id = (int)$row['id'];
22        $user->username = $row['username'];
23        $user->password_hash = $row['password_hash'];
24
25        return $user;
26    }
27 }
```

Modelo Product:

```
1 <?php
2 declare(strict_types=1);
3
4 class Product
5 {
6     public int $id;
7     public string $name;
8     public float $price;
9     public string $created_at;
10
11     /**
12      * Obtiene todos los productos.
13      */
14     public static function all(PDO $pdo): array
15     {
16         $stmt = $pdo->query('SELECT * FROM products ORDER BY id DESC');
17         return $stmt->fetchAll(PDO::FETCH_ASSOC);
18     }
19
20     /**
21      * Busca un producto por ID.
22      */
23     public static function find(PDO $pdo, int $id): ?array
24     {
25         $stmt = $pdo->prepare('SELECT * FROM products WHERE id = :id');
26         $stmt->execute([':id' => $id]);
27         $row = $stmt->fetch(PDO::FETCH_ASSOC);
28
29         return $row ? : null;
30     }
31
32     /**
33      * Crea un nuevo producto.
34      */
35     public static function create(PDO $pdo, string $name, float $price): void
36     {
37         $stmt = $pdo->prepare('
38             INSERT INTO products (name, price, created_at)
39             VALUES (:name, :price, :created_at)
40         ');
41         $stmt->execute([
42             ':name' => $name,
43             ':price' => $price,
44             ':created_at' => date('Y-m-d H:i:s'),
45         ]);
46     }
47
48     /**
49      * Actualiza un producto existente.
50      */
```


Controlador AuthController:

AuthController.php · líneas 1-40

```
1 <?php
2 declare(strict_types=1);
3
4 class AuthController
5 {
6     private PDO $pdo;
7
8     public function __construct(PDO $pdo)
9     {
10         $this->pdo = $pdo;
11     }
12
13     /**
14      * Muestra el formulario de login.
15      */
16     public function login(): void
17     {
18         $error = $_GET['error'] ?? null;
19         require __DIR__ . '/../views/auth/login.php';
20     }
21
22     /**
23      * Procesa el login.
24      */
25     public function doLogin(): void
26     {
27         ensureSession();
28
29         $username = trim($_POST['username'] ?? '');
30         $password = trim($_POST['password'] ?? '');
31
32         if ($username === '' || $password === '') {
33             $error = 'Debes rellenar usuario y contraseña.';
34             require __DIR__ . '/../views/auth/login.php';
35             return;
36         }
37
38         $user = User::findByUsername($this->pdo, $username);
39
40         if (!$user || !password_verify($password, $user->password_hash)) {
```

Login en el navegador:

Iniciar sesión. CRUD Francisco

Usuario

Contraseña

Entrar

Usuario por defecto: **admin** — Contraseña: **admin**
(primero ejecuta create_db.php).

Ejemplo MVC PHP + SQLite + Login con hash de contraseña.

Conclusiones

Lo que me llevo de esta unidad:

- HTTP es la base de toda la web, y entender qué viaja en una petición y en una respuesta es lo que me ha permitido empezar a "ver" lo que pasa entre el navegador y el servidor.
- Las **cabeceras** son la herramienta para casi todo: cambiar el tipo de contenido, devolver errores, redirigir, controlar la caché...
- **JSON** es el formato estándar para intercambiar datos. Cualquier API moderna devuelve JSON.
- Las **sesiones** y las **cookies** sirven para mantener estado, que es lo que HTTP por sí solo no hace.
- Para las **contraseñas** siempre `password_hash` + `password_verify`. Nunca en claro, nunca con MD5 o SHA1.
- El **MVC** ordena el código en proyectos grandes: el front controller decide qué hacer, los controladores procesan, los modelos hablan con la BD y las vistas se encargan de mostrar.

Esta unidad es donde por primera vez se monta algo que se parece a una aplicación web "de verdad", con varios usuarios, base de datos y sesiones. Es de las más útiles del curso.